

2.2 Image Tagging Strategy, Production Tracking and Rollback Procedure

Image Tagging Strategy

This project uses semantic versioning (SemVer) for Docker images rather than relying on the mutable `latest` tag. Every application component is built and published with an explicit version number.

Images are stored in Docker Hub using the following naming convention:

- `kunzydev/formflow-backend:1.0.0`
- `kunzydev/formflow-frontend:1.0.0`
- `kunzydev/formflow-nginx:1.0.0`

Whenever a new release is produced, the version number is incremented.

Example releases include:

Component	Previous	Current
Backend	1.0.0	1.0.1
Backend	1.0.1	1.0.2
Frontend	1.0.0	1.0.1
Nginx	1.0.0	1.0.1

No Docker image is deployed using the `latest` tag. This guarantees that every deployment is deterministic and reproducible.

Production Image Selection

The production deployment does not specify image versions directly inside `docker-compose.prod.yml`.

Instead, all services reference a shared environment variable:

```
image: kunzydev/formflow-backend:${IMAGE_TAG}
```

The `.env.production` file contains the currently deployed release.

Example:

```
IMAGE_TAG=1.0.2
```

Changing the value of `IMAGE_TAG` changes the version of every application component that Docker Compose pulls and deploys.

This makes the environment configuration the single source of truth for the deployed application version.

Determining the Live Production Version

At any point, the currently deployed release can be identified using several methods.

1. Inspect the production environment file:

```
cat .env.production
```

Example:

```
IMAGE_TAG=1.0.2
```

1. Inspect the running container:

```
docker inspect formflow-backend --format='{{.Config.Image}}'
```

Example output:

```
kunzydev/formflow-backend:1.0.2
```

1. List all running containers:

```
docker ps
```

The image column clearly shows which tagged release is running.

These approaches ensure that the deployed production version is always known and can be verified without ambiguity.

Rollback Procedure

If a newly deployed release introduces defects, the application can be restored to the previous stable version using the following procedure.

Step 1

Edit the production environment file.

Change:

```
IMAGE_TAG=1.0.2
```

to

```
IMAGE_TAG=1.0.1
```

Step 2

Pull the previous release from Docker Hub.

```
docker compose
--env-file .env.production
-f docker-compose.prod.yml
pull
```

Step 3

Restart the containers.

```
docker compose
--env-file .env.production
-f docker-compose.prod.yml
up -d
```

Step 4

Verify the running version.

```
docker inspect formflow-backend --format='{{.Config.Image}}'
```

Expected output:

```
kunzydev/formflow-backend:1.0.1
```

This rollback process requires no image rebuilding because all previously released versions remain stored in Docker Hub. Recovery therefore takes only a few minutes and minimizes application downtime.

2.3 Secrets Handling Plan

The FormFlow application requires several sensitive configuration values during development and production. These include database credentials, application configuration and Docker registry credentials used by the CI/CD pipeline.

Required Secrets

Secret	Purpose	Storage Location
POSTGRES_USER	PostgreSQL username	<code>.env.production</code> on deployment server
POSTGRES_PASSWORD	PostgreSQL password	<code>.env.production</code> on deployment server
POSTGRES_DB	Database name	<code>.env.production</code>
DATABASE_URL	Prisma database connection string	<code>.env.production</code>
NODE_ENV	Runtime configuration	<code>.env.production</code>
PORT	Backend application port	<code>.env.production</code>
NEXT_PUBLIC_API_URL	Frontend API endpoint	<code>.env.production</code>
IMAGE_TAG	Production image version	<code>.env.production</code>
DOCKER_USERNAME	Docker Hub username	GitHub Actions Repository Secret
DOCKER_TOKEN	Docker Hub Access Token	GitHub Actions Repository Secret
AZURE_VM_IP	Deployment target	GitHub Actions Repository Secret
AZURE_VM_USER	SSH username	GitHub Actions Repository Secret
AZURE_VM_SSH_KEY	SSH private key	GitHub Actions Repository Secret

Secret Storage Strategy

Sensitive information is never hardcoded into the application source code, Dockerfiles or GitHub repository.

Environment-specific values are stored in `.env.production` on the deployment server, while CI/CD credentials are stored as encrypted GitHub Actions Repository Secrets.

During deployment, Docker Compose automatically injects these values into the running containers using the `env_file` directive.

This separation ensures that sensitive information is isolated from the application source code and can be changed without rebuilding Docker images.

Security Measures

The following practices are implemented to protect sensitive information throughout the deployment process.

- `.env` files are excluded from version control using `.gitignore`.
- Dockerfiles contain no passwords, tokens or connection strings.
- GitHub Actions accesses secrets only through encrypted repository secrets.
- Docker Hub authentication uses a Personal Access Token instead of an account password.
- Production image versions are managed through the `IMAGE_TAG` environment variable rather than modifying application code.
- Previous Docker image versions remain available in Docker Hub, enabling safe rollback without rebuilding images.

This approach follows common DevOps practices for secure secret management while supporting repeatable and reliable deployments.